UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

TESI DI LAUREA

A SOTIF-compliant Framework for the Evaluation of Scenario Generation Tools in ADS Co-Simulation

RELATORE

Prof. Dario Di Nucci

Dott. Antonio Trovato

Università degli Studi di Salerno

CANDIDATO

Nicola Laurino

Matricola: 0522501438

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

And so you touch this limit, something happens and you suddenly can go a little bit further

— Ayrton Senna

Abstract

Automated Driving Systems (ADS) automate perception, planning, and vehicle control, and therefore require extensive validation, especially in rare and critical conditions. However, on-road testing is often dangerous and costly, and it does not allow systematic coverage of the wide variety of possible situations. For this reason, validation shifts to simulation and co-simulation environments, where tests are controlled and repeatable, but depend on the availability of driving scenarios: manually crafting them is expensive and not scalable. Consequently, AI-based automatic scenario generation tools (such as TM-Fuzz, SimADFuzz, and ScenarioFuzz-LLM) are increasingly adopted, yet they still lack a standard-compliant evaluation process aligned with ISO 21448 (SOTIF), making rigorous comparisons difficult. This thesis delivers a SOTIF-compliant, generalizable evaluation framework for scenario generators and applies it to an empirical comparison of state-of-the-art tools in CARLA-based co-simulation. For each scenario, multiple runs are executed in CARLA, producing logs that are enriched to collect safety, functional, and etiquette metrics, estimate residual risk against acceptance criteria, and evaluate generator efficiency. The resulting pipeline enables a reproducible, SOTIF-aligned assessment of scenario generators in terms of observed criticalities, coverage, diversity of non-acceptable cases, and efficiency in producing safety-relevant scenarios. Overall, the results highlight clear differences across the tools: ScenarioFuzzLLM generates, on average, more critical scenarios and is the most efficient in eliciting collisions within shorter times, whereas TMFuzz demonstrates the strongest exploratory capability in terms of diversity and coverage of hazardous behaviours. Some driving-style hazards occur very frequently under the adopted experimental setup and are therefore less discriminative, while the most meaningful differences among tools primarily emerge on rarer events and on the time required for hazards to manifest.

Contents

List of Figures

List of Tables

CHAPTER 1

Introduction

1.1 Application Context

Modern transportation systems are progressively incorporating software components and automated decision-making capabilities, evolving into cyber-physical systems deployed within an inherently safety-critical domain such as mobility. Within this context, Advanced Driver Assistance Systems [1] (ADAS) and Automated Driving Systems [2] (ADS) represent the automotive industry's evolutionary trajectory toward increasing levels of driving automation. In particular, ADAS provide driver support functions and partial automation (e.g., lane keeping assistance or automatic emergency braking), whereas ADS aim to perform broader portions of the dynamic driving task, up to advanced levels of automation.

The introduction and widespread adoption of these systems is closely linked to the need to improve road safety, mitigate the impact of accidents, and enhance driving efficiency and comfort. However, the urgency of the problem is supported by consolidated evidence: the World Health Organization estimates that approximately 1.19 million people die each year due to road traffic crashes, and that traffic-related injuries are among the leading causes of mortality in the young population, ranking

as the leading cause of death in the 5–29 age group[1]. In the European context, despite an overall improving trend, the burden remains high: according to data published by the European Commission, in 2024 the European Union recorded 19,940 road traffic fatalities, with a slight decrease compared to 2023 [3]. In this scenario, ADAS and ADS are commonly presented as technologies capable of contributing to accident reduction; nevertheless, their deployment makes a central issue unavoidable: how to ensure their safety and reliability under real-world conditions.

The safety of automated driving systems cannot be reduced solely to the prevention of hardware failures or software defects. A substantial portion of risk stems from situations in which the system operates as specified, yet remains unsafe due to functional insufficiencies (e.g., perceptual or decision-making limitations). To address this class of risks, the ISO 21448:2022 standard, known as SOTIF (Safety of the Intended Functionality), provides a formal framework for the analysis of hazards not attributable to traditional faults and requires a risk-based assessment grounded in key concepts such as the Operational Design Domain (ODD), triggering conditions, hazardous scenarios, and the estimation of residual risk against acceptance criteria.

Meeting these requirements exclusively through on-road testing is widely problematic: conducting tests in public environments entails safety risks and high costs and, crucially, does not allow the controlled and systematic reproduction of the wide range of rare yet critical conditions needed to demonstrate an acceptable residual risk. For these reasons, ADS/ADAS validation heavily relies on co-simulation environments, which enable scalable, repeatable, and controlled testing while reducing costs and risks. In both the literature and experimental practice, CARLA[2] is among the most widely adopted simulators, providing photorealistic urban environments and flexible APIs for integrating perception and planning modules. Nevertheless, simulation alone is not sufficient: the quality of the assessment depends directly on the quality of the test scenarios. Manually designing realistic scenarios in CARLA requires specialized expertise and is often a costly, error-prone, and poorly scalable process, negatively affecting reproducibility and the breadth of experimental coverage. Consequently, automated scenario generation tools have been proposed to

[1] World Health Organization - Road traffic injuries

[2] CARLA's project is available at `https://carla.org/`.

systematically produce larger, more diverse, and potentially more critical test suites.

1.2 Motivation and Objectives

The increasing adoption of ADS/ADAS makes it necessary to rely on validation processes that do not merely verify the absence of faults, but also enable the identification and quantification of hazards arising from functional insufficiencies, in line with the SOTIF approach. From this perspective, validation should be able to identify the triggering conditions that may activate functional insufficiencies and to support an estimation of residual risk against acceptance criteria. A second motivating factor is that, in practice, ADS/ADAS assessment largely relies on co-simulation and scenario-based testing. To address the burden of manual scenario design, numerous automated scenario generation tools have emerged with the goal of producing large, systematic, and diverse test suites. However, this evolution raises a key methodological issue: the quality of the generated scenarios directly affects the validity of the assessment. Generators that fail to produce sufficiently hazardous, diverse, or efficient scenarios may lead to misleading outcomes, for instance by conveying a perception of safety that is not supported by experimental evidence. This motivates the need for an empirical comparison among scenario generators that is rigorous and aligned with standards.

This thesis aims to conduct an empirical comparison of state-of-the-art automated scenario generation tools for ADS validation in CARLA, by introducing and applying an evaluation framework explicitly aligned with ISO 21448:2022 (SOTIF). The framework integrates ODD specification, triggering-condition analysis, and residual-risk estimation into the assessment process. This thesis provides the following contributions:

- an empirical comparison of three state-of-the-art automated scenario generation tools in a CARLA-based co-simulation setting, conducted on a common open-source ADS baseline through repeated executions and a uniform logging and analysis pipeline, enabling a fair assessment in terms of safety relevance, diversity, and efficiency;

- an end-to-end, reusable evaluation pipeline aligned with SOTIF that operationalizes ODD characterization, hazard and residual-risk estimation, and diversity analysis, and produces comparable outputs (metrics and reports) to support systematic comparisons across generators under the same experimental conditions.

1.3 Summary of Results

This section presents the main findings from the comparison of three automated scenario generation tools in simulation (ScenarioFuzzLLM, SimADFuzz, and TM-Fuzz), with the goal of assessing their effectiveness, diversity, and efficiency in producing safety-relevant scenarios. The analysis showed that the tools differ both in terms of the average criticality of the generated scenarios and in the types of hazards they tend to elicit. ScenarioFuzzLLM produced, on average, riskier scenarios and triggered collisions more quickly and consistently, whereas TMFuzz exhibited stronger exploratory capability and a more uniform distribution of hazardous run types. SimADFuzz showed more variable results and an intermediate profile, with more pronounced differences for specific event classes. Finally, some driving-style hazards (e.g., lane invasion and off-road) are highly frequent under the adopted experimental setup and should be interpreted as indicators of instability rather than rare events; consequently, the most meaningful distinctions across tools emerge from diversity metrics and from the time required to elicit the most informative hazards.

1.4 Thesis Structure

This thesis is organized into six chapters, each focusing on a specific aspect.

Chapter 1 – Introduction. This chapter introduces the application context of automated driving and scenario-based validation, outlining the core concepts related to SOTIF, the limitations of real-world testing, and the role of co-simulation. It also presents the motivation and objectives of the study, together with a concise summary of the results obtained.

Chapter 2 – Background and state of the art. This chapter provides the theoretical foundations needed to frame the problem: ADS/ADAS and simulation-based validation; key concepts from ISO 21448 (ODD, triggering conditions, hazardous scenarios, residual risk); and a critical review of the main existing approaches and benchmarks for scenario generation and evaluation, highlighting limitations and gaps with respect to SOTIF-aligned comparisons.

Chapter 3 – Research methodology. This chapter describes in detail the proposed evaluation framework and the experimental pipeline, including ODD definition, scenario annotation procedures, CARLA simulator configuration, repeated-execution protocols, log extraction, and metric computation. It also formalizes the comparison criteria adopted for the scenario generators, including design choices related to residual-risk estimation and to coverage and diversity measures.

Chapter 4 – Results analysis. This chapter reports the experimental results obtained through co-simulation and presents the empirical comparison of the scenario generators according to the framework dimensions. The analysis integrates indicators of criticality and risk, coverage of the ODD and triggering conditions, diversity of non-acceptable scenarios, and computational efficiency, discussing the observed patterns and the main findings.

Chapter 5 – Threats to validity. This chapter discusses the main threats to validity and the strategies adopted to mitigate experimental bias, simulator limitations, dependencies on the considered ADS implementations, and the sensitivity of metrics and ranking criteria.

Chapter 6 – Conclusions. This chapter summarizes the main contributions and evidence, relates the results to the initial objectives, discusses practical implications for scenario-based validation and for SOTIF-oriented comparison of scenario generators, and outlines directions for future work.

CHAPTER 2

Background and state of the art

This chapter provides the concepts and references needed to frame the validation of ADAS/ADS systems in simulation and the role of automated scenario generation. It introduces the fundamental principles of automation levels and the SAE taxonomy, together with the key concepts of ISO 21448 (SOTIF), with particular attention to the concept of ODD, triggering conditions, and residual risk. It then discusses validation approaches based on real-world testing and on simulated environments, motivating the adoption of co-simulation and presenting CARLA as a reference simulator. In the second part, the chapter describes automated scenario generation and analyses the main tools considered, highlighting how the current state of the art and existing benchmarks only support partial comparisons, and motivating the need for a SOTIF-aligned empirical evaluation of scenario generators.

2.1 ADS/ADAS

Advanced Driver Assistance Systems (ADAS) [1] and Automated Driving Systems [2] (ADS) represent one of the major technological developments in the automotive domain. ADAS introduce driver support functionalities and partial automation of the driving task (e.g., adaptive cruise control, lane keeping assistance, automatic emergency braking), with the aim of assisting the driver in handling specific maneuvers or operating conditions. ADS, by contrast, target higher levels of automation, up to fully autonomous driving, progressively taking over the execution of the driving task within specific contexts. From a conceptual standpoint, these systems fall within the category of cyber-physical systems, as they integrate computational components (perception, decision-making, and control) with a physical process (vehicle dynamics and interaction with the road environment), operating in a safety-critical domain.

To distinguish ADAS and ADS unambiguously, both the literature and standards adopt the concept of the Dynamic Driving Task [4] (DDT), namely the set of operational activities required for driving (lateral and longitudinal control, environment monitoring, event response, and short-horizon planning). In driver assistance systems, the user typically remains responsible for the overall DDT; in automated driving systems, instead, the DDT is performed by the system (fully or partially, under clearly specified constraints).

The most widely used reference for classifying the degree of driving automation is the SAE J3016 taxonomy [5], which defines six levels based on who performs the Dynamic Driving Task (DDT) and who is responsible for the fallback in the presence of boundary conditions or functional degradation, as follows:

- *L0 (No Automation).* The system may provide warnings or momentary assistance, but it does not continuously perform vehicle control.
- *L1 (Driver Assistance).* Automation of a single control component (either lateral or longitudinal), with continuous driver supervision.
- *L2 (Partial Driving Automation).* Combined automation of both lateral and longitudinal control under certain conditions, while requiring continuous supervision and keeping responsibility with the driver.

- *L3 (Conditional Driving Automation).* The system performs the DDT within a defined domain; the driver is not required to supervise continuously, but must be available to intervene upon the system's request (fallback demand).
- *L4 (High Driving Automation).* The system performs the DDT and manages fallback within a defined ODD; outside the ODD, the system must reach a minimal-risk condition without relying on immediate human intervention.
- *L5 (Full Driving Automation).* The system performs the DDT and fallback under all operating conditions, without ODD constraints.

SAE J3016 provides only a functional taxonomy (terminology and operational responsibilities), rather than a direct criterion for safety or regulatory compliance. Nevertheless, it is widely adopted because it enables a consistent description of "who does what" in driving automation, and is therefore useful as a conceptual foundation for any discussion on verification, validation, and safety assurance[6].

2.2 ISO 21448:2022

Safety verification for ADS/ADAS cannot be addressed exclusively through the traditional functional safety paradigm, which focuses on faults and malfunctions of electrical/electronic (E/E) components and software. ISO 26262 [7] specifically targets risks associated with hazards caused by the malfunctioning behaviour of safety-related E/E systems. However, a significant portion of risk can also arise when the system is not faulty yet remains unsafe due to functional insufficiencies (such as limitations in perception, interpretation, or planning) or as a result of reasonably foreseeable misuse. To address this class of risks, ISO 21448:2022 [8], known as SOTIF (Safety of the Intended Functionality), provides a methodological framework aimed at ensuring the "absence of unreasonable risk" due to hazards resulting from functional insufficiencies of the intended functionality, or from foreseeable misuse, in the absence of hardware/software faults.

Within the scope of this work, SOTIF is the central reference for evaluating scenario generation tools in a standard-aligned manner, linking test generation to the

notion of risk and to acceptance criteria. Among the fundamental concepts introduced by SOTIF, three are particularly relevant for scenario-based validation.

1. *Operational Design Domain (ODD).* The set of environmental, infrastructural, traffic-related, and operational conditions within which an ADS/ADAS is designed to operate safely (e.g., lighting, weather, road type and geometry, traffic density/composition, speed).

2. *Triggering Conditions.* Combinations of factors, events, or configurations that may activate a functional insufficiency and bring the system into a hazardous state. They do not necessarily coincide with extreme conditions: they may include perceptual ambiguities, partial occlusions, unexpected yet plausible behaviours of other road users, or complex interactions among multiple actors. In a scenario-based setting, identifying triggering conditions is crucial because they link scenario characteristics to causal mechanisms that may lead to hazards.

3. *Hazardous scenarios.* Operational situations in which, given a specific portion of the ODD and one or more triggering conditions, the system's behaviour may produce a hazardous outcome (e.g., collision, road departure, or critical violation of traffic rules). From a SOTIF perspective, scenario-based assessment goes beyond counting "how many accidents" occur, and instead aims to build a structured mapping between ODD, triggering conditions, and hazard-relevant scenarios to support repeatable measures of coverage and risk.

SOTIF follows a risk-based approach, whose objective is to estimate and reduce the residual risk associated with the intended functionality until it reaches an acceptable level according to defined criteria. From this standpoint, effective validation must link the outcomes observed during testing to an assessment that accounts for both the probability of hazardous events and their severity.

In the context of simulation, probability estimation plays a particularly important role because many elements of the system and the environment are (or can be modelled as) stochastic: sensor noise, non-deterministic behaviours of traffic agents, and variability introduced by the scenario generator. As a result, a scenario cannot be

reliably assessed through a single execution; repeated runs are required to empirically estimate how frequently the scenario produces a hazardous outcome.

Combining the empirical probability of the outcome with a measure of severity makes it possible to derive a scenario-level risk estimate and, by aggregating such information, an overall residual-risk measure.

The adoption of automated scenario generators makes it necessary to establish methodologically sound comparison criteria. In particular, if a generator produces a large number of scenarios that are poorly aligned with the ODD and triggering conditions, or if it discovers few critical cases and does so inefficiently, the evaluation may become biased and lead to misleading conclusions, including a perception of safety that is not supported by sufficient evidence.

2.3 Real-World Testing and Simulated Environments

ADAS/ADS validation can be carried out through on-road experimentation (*real-world testing*) and through simulated environments, often organised according to a scenario-based approach. In principle, real-world testing provides direct exposure to the complexity of real traffic; in practice, however, it presents structural limitations that reduce its sustainability as the primary strategy for the safety assurance of advanced systems.

The first limitation concerns cost and risk: conducting extensive on-road testing requires operational infrastructure, personnel, instrumented vehicles, and safety procedures, with constraints that grow rapidly as the ambition of the ADS increases.

The second limitation is methodological and more critical: the difficulty of obtaining statistically robust evidence on rare but high-severity events (serious crashes, injuries, fatalities). Kalra and Paddock [9] show that, precisely because fatalities and injuries are relatively rare compared to vehicle-kilometres travelled, demonstrating with adequate confidence the safety of an autonomous vehicle through real-world driving alone would require hundreds of millions of miles and, in some cases, hundreds of billions of miles travelled, making this approach impractical as a sole pre-deployment strategy. These considerations are particularly relevant in a SOTIF context: many functional insufficiencies emerge under rare conditions, and

opportunistic exposure to real traffic alone does not guarantee either systematic coverage or reproducibility.

To address these limitations, adopting simulated environments allows for transferring a substantial portion of validation into settings that are controllable, repeatable, and scalable, thereby reducing costs and risks. In a simulated environment, it is possible to exercise fine-grained control over scenario parameters, systematically reproducing rare or hard-to-observe conditions in the real world, such as adverse weather, sensor occlusions, or complex interactions among multiple actors. Moreover, simulation enables the same scenario to be repeated many times, making it possible to estimate the effect of stochastic components and to reduce dependence on potentially anomalous single executions. Finally, thanks to automated execution and scenario parametrisation, large and diverse test datasets can be generated without relying on randomness or on the availability of specific real-world traffic conditions.

Single-simulator simulation refers to an approach in which a single simulation environment models, with a predefined level of fidelity, elements such as the 3D world, vehicle dynamics, traffic, and sensor simulation. This approach is particularly useful when automated and repeatable workflows are required to execute large volumes of scenarios, but it remains inevitably constrained by the modelling assumptions and trade-offs of the chosen simulator.

Co-simulation [10] extends this paradigm by combining multiple simulators or specialised models, each optimised for a sub-domain, with data exchange during execution. This enables, for example, coupling a 3D environment simulator (e.g., for sensor simulation and rendering) with a traffic simulator and a more realistic vehicle-dynamics model, resulting in an environment that is overall more aligned with validation objectives.

Compared to single-simulator simulation, co-simulation aims to improve the quality of the testbed through model complementarity, reducing dependence on a single backend and enabling more robust analyses in the presence of fidelity differences across components. Moreover, multi-model variants (e.g., combining traffic simulation and vehicle dynamics with different levels of detail depending on the distance from the ego vehicle) are used to increase scalability while maintaining high fidelity where it is most relevant [11].

2.4 CARLA

CARLA[1] (Car Learning to Act) is an open-source simulator introduced in 2017 to support research and development in autonomous driving, with an explicit emphasis on the development, training, and validation of ADS in urban scenarios. The seminal work presented CARLA as a platform designed "from the ground up" for controlled experimentation, including open-source code, open protocols, and freely usable digital assets (urban maps, buildings, vehicles) [12].

The main motivation behind CARLA is to enable systematic experimentation in a domain where data collection and validation exclusively on public roads are costly and poorly scalable. From this perspective, CARLA is designed to enable the configuration of controlled scenarios with progressively increasing levels of complexity, supporting systematic and repeatable experimentation. In addition, the platform makes it possible to evaluate different approaches through reproducible metrics, fostering consistent comparisons across methods and configurations. Finally, thanks to its APIs and modular architecture, CARLA facilitates the integration of components typical of autonomous driving pipelines—particularly perception and planning modules—making it well suited for automated development and validation workflows.

This setup makes CARLA particularly suitable for scenario-based testing pipelines and, more generally, for strategies that require automation in scenario generation and execution. From a functional standpoint, CARLA provides features that make it well suited as an experimental backend:

- **Control of static and dynamic actors.** The ability to define and manipulate vehicles, pedestrians, and infrastructure elements (road layout, traffic lights), making scenarios explicitly parameterisable.

- **Flexible sensor-suite configuration.** Support for virtual sensors and heterogeneous configurations to assess perception-related effects and to construct scenarios that include degraded or ambiguous conditions (a key aspect in SOTIF).

[1]The project is available at https://carla.org/.

- **Variability of environmental conditions.** Configuration of weather/lighting and other environmental factors, enabling exploration of ODD regions and potential triggering conditions.
- **Integration with automated workflows.** Being open-source and programmable, CARLA lends itself to large-scale and repeated experiments, as typically required by pipelines for comparing scenario generators.

2.5 BeamNG.tech

BeamNG.tech[2] is a simulator used in both academic and industrial settings as a backend for scenario-based testing of driving systems, with a strong emphasis on vehicle-dynamics fidelity and physical modelling of vehicles and collisions [13]. Unlike simulators primarily oriented towards photorealistic urban environments and sensor simulation, BeamNG.tech is often selected when the experimental objective requires fine-grained control over road geometry, vehicle behaviour, and physical interactions, and thus represents a natural complement within a co-simulation setting.

Motivations for Selecting CARLA as the Experimental Backend The experimental implementation of this thesis was conducted using CARLA as the backend. This choice is driven by implementation and comparability constraints: the selected generators and the logging pipeline were integrated and validated end-to-end on CARLA, whereas extending the setup to BeamNG.tech would have required an additional engineering layer, in particular the automatic conversion of scenarios across formats and the adaptation of logging to the APIs and data structures of the second simulator. To preserve experimental coherence and complete the full SOTIF-aligned pipeline within the project constraints, the evaluation was therefore focused on the CARLA environment only.

[2]The project is available at https://beamng.tech/

2.6 Scenario Generation Tools

Simulation-based validation of ADS/ADAS is typically organised according to a scenario-based paradigm, in which the system is assessed through a set of synthetic yet controlled driving situations. In this context, the term scenario refers to a structured description of initial and dynamic conditions that determines how the interaction between the ego vehicle and the environment unfolds. This perspective is widely recognised as a key element in the development, testing, and validation processes for automated driving systems [14]. A scenario generator is a tool that automatically produces executable scenario descriptions for simulation, often by exploring a very large and highly combinatorial configuration space. The primary motivation for adopting automated scenario generators is the need to overcome the limitations of manual scenario design. Manually defining a large number of realistic and technically valid scenarios requires expertise, time, and significant verification effort, negatively affecting scalability, reproducibility, and coverage. The literature proposes several families of approaches, which are often combinable:

- *Template-based [15].* Scenarios derived from templates or catalogues (often inspired by crashes or recurring patterns). They offer control and interpretability, but may suffer from rigidity and limited coverage when templates are few or overly specific.

- *Search-based [16].* Scenarios generated as the solution to a search/optimisation problem in a parametric space (e.g., maximising collision likelihood or rule violations). These approaches can produce "targeted" scenarios, but their effectiveness depends strongly on the objective function and constraints.

- *Fuzzing-based and simulation-feedback fuzzing [17].* Approaches that iteratively mutate scenarios and select "interesting" ones based on simulation feedback (violations, criticality signals, novelty/diversity).

- *LLM-guided [18].* Approaches that leverage language models or foundation models to guide scenario construction or semantic enrichment, for instance by increasing diversity and the variety of plausible interactions.

These families share the idea that automated generation serves as a means to explore the scenario space in order to maximise coverage and the discovery of critical cases, while preserving realism constraints and executable validity [19].

2.7 State of the Art

ADS/ADAS validation must be scenario-based, repeatable, and scalable. At present, obtaining "fair" comparisons across different scenario generation approaches remains difficult due to differences in simulation platforms, scenario sets, metrics, and experimental protocols. This work positions its contribution in proposing an empirical comparison of scenario generators that is reproducible and, crucially, aligned with SOTIF concepts (ODD, triggering conditions, residual risk).

SafeBench SafeBench[3] is a benchmarking platform built on CARLA with the goal of providing a unified context for evaluating the robustness and safety of autonomous driving algorithms under safety-critical scenarios, integrating scenarios, generation mechanisms, and evaluation procedures within a common workflow. The authors motivate SafeBench by noting that, despite the substantial body of work on autonomous driving testing, it remains difficult to compare the effectiveness and efficiency of different scenario generation and testing approaches under comparable conditions. The platform proposes a modular architecture and a reusable scenario library to support systematic evaluations, thereby simplifying comparisons across agents and testing mechanisms within a controlled environment [20]. Although SafeBench represents a relevant CARLA-based benchmark, it is not designed to be standard-aligned with ISO 21448, as it does not structure the comparison by explicitly requiring ODD characterisation, systematic tracking of triggering conditions, and residual-risk estimation.

SBFT CPS Testing Tool Competition Dedicated to self-driving cars in BeamNG.tech, the SBFT CPS Testing Tool Competition is an initiative explicitly aimed at the empirical comparison of testing tools. The competition is framed as a test-generation

[3]The project is available at https://safebench.github.io/

challenge based on virtual roads, where the goal is to trigger failure behaviours related to the ability of the system under test to keep the vehicle within lane boundaries (lane keeping / lane keeping assist), by generating roads that induce deviations or lane departures. From a methodological standpoint, the SBFT competition is particularly valuable because it defines shared rules, test validity constraints, and a common experimental budget, thereby enabling a direct comparison across tools [21]. The SBFT Tool Competition (Cyber-Physical Systems Track) is organised as an annual challenge focused on testing a Lane Keeping Assist System (LKAS) in the BeamNG.tech environment. On the one hand, tools are evaluated in terms of their ability to generate failure-inducing tests and, importantly, the competition explicitly considers diversity-related aspects. On the other hand, the scope remains necessarily narrow (a specific system under test and a specific input space based on road generation), and it does not fully cover the elements required for a SOTIF-aligned assessment, such as systematic coverage of the ODD and triggering conditions, nor residual-risk estimation over a broader set of hazardous outcomes.

CHAPTER 3

Research Methodology

This chapter describes the experimental design and the full methodological pipeline adopted to compare scenario generators in a repeatable and comparable manner. After defining the scope of the analysed tools, it presents the procedure for executing scenarios in CARLA, including the adoption of a uniform driving agent and the logging system used to record the mission, frame-by-frame states, and events. The chapter then outlines the log enrichment phase and the implemented SOTIF pipeline to extract aggregated metrics and final reports. Finally, it formalises the metrics and scores used to answer the research questions, clarifying how experimental outputs are transformed into indicators that are comparable across tools.

3.1 Research Questions

The research questions guiding this work were defined with the aim of comparing different automated scenario generation tools in simulation, assessing not only their ability to produce hazardous situations, but also the variety of the criticalities observed and the speed at which such criticalities emerge. The questions were formulated to steer the analysis along four complementary directions: effectiveness, diversity, non-critical driving behaviours, and efficiency.

🔍 **RQ$_1$.** *How effective are different scenario generation tools in producing hazardous scenarios?*

This research question assesses the extent to which the different tools are able to generate scenarios that include safety-relevant hazardous conditions. The aim is to compare the generators in terms of their capability to elicit overall riskier scenarios and to highlight any systematic differences across the tools.

🔍 **RQ$_2$.** *Do scenario generation tools differ in the diversity of hazardous scenarios they reveal?*

This research question investigates whether the tools generate hazardous scenarios that are "similar" to one another or, conversely, whether they tend to explore different types of hazardous situations. The objective is to assess the diversity of critical cases identified by each generator and to determine whether some tools exhibit a more exploratory behaviour than others.

🔍 **RQ$_3$.** *Do scenario generation tools differ in the occurrence of non-critical driving-related hazards?*

This research question focuses on non-critical hazards understood as indicators of improper driving behaviour, namely events that do not correspond to collisions but still represent violations or vehicle instability. In particular, the analysis considers red-light running, stop-sign violations, lane invasions, and road departures. The goal is to determine whether the tools differ in the type and frequency of these events, outlining generation profiles that are more oriented towards violations than others.

🔍 **RQ$_4$.** *Which scenario generation tool reveals SOTIF-relevant hazards more efficiently?*

This research question introduces the efficiency dimension, understood as the ability to elicit safety-relevant hazards within the shortest "useful time" during simulation

execution, under the same time budget. The objective is to determine whether some tools enable earlier observation of potentially hazardous conditions, and are therefore better suited for building effective and time-efficient test suites.

3.2 Analysed Tools

The literature on automated scenario generation for ADS/ADAS proposes a variety of approaches, commonly grouped into families such as template-based, search-based, fuzzing-based, and LLM/foundation-model-guided approaches, as discussed in Chapter 2.3 (Scenario Generation Tools) [22].

Consistently with the goal of this work—namely, an empirical comparison of *state-of-the-art* generators through a SOTIF-aligned framework—the tool selection was guided by criteria of experimental eligibility and methodological representativeness. In line with the methodological indications of the registered report, priority was given to tools that produce scenarios suitable for ADS/ADAS testing in simulation and that are compatible with co-simulation backends, in particular CARLA. Accordingly, the following operational criteria were adopted:

- compatibility with CARLA and end-to-end integration within the experimental pipeline (repeated execution, logging, enrichment, and metric computation);
- availability of an executable and documented implementation, preferring open-source projects or accessible artefacts;
- experimental stability, intended as the ability to generate valid and executable scenarios consistently, reducing failures due to unmaintained tools;
- representativeness within the state of the art, favouring recent tools proposed in scientific venues or described in technical papers, which embody different strategies for exploring the scenario space [19].

The analysis focused on ScenarioFuzz-LLM, TM-fuzzer, and SimADFuzz, as they satisfy the experimental constraints (CARLA compatibility, executability, integrable logging) while also covering three distinct generation modalities within

the fuzzing/search-guided paradigm, making the comparison more meaningful. Specifically, the three tools were selected because:

- they can all be framed as automated scenario-based testing with fuzzing/search logic guided by feedback, which is the class most consistent with the objective of generating hazard-relevant scenarios in a systematic and scalable manner;
- they differ in their exploration strategy, reducing the risk of comparing three near-identical variants of the same approach.

ScenarioFuzz-LLM ScenarioFuzz-LLM[1] introduces a distinctive element compared to traditional fuzzers: the use of a Large Language Model to support scenario generation and variation, with the explicit goal of increasing the diversity of the produced configurations [23]. In practice, the LLM acts as a "semantic" guidance component that proposes plausible variations (e.g., in actor configurations or scenario conditions), reducing the risk of generating mutations that are of limited significance or highly repetitive [24].

TM-fuzzer TM-Fuzzer[2] belongs to the class of mutation-oriented fuzzers that perform iterative exploration of the scenario space, and is built around two key ideas:

- real-time traffic management, i.e., dynamically manipulating NPC behaviour or traffic flow to increase the likelihood of critical interactions with the ego vehicle;
- a diversity analysis component to prevent the search process from converging to scenarios that are highly similar to one another.

The tool aims to generate *security-/safety-critical* scenarios while also ensuring that the produced cases remain unique within a potentially vast search space [25].

SimADFuzz SimADFuzz[3] adopts a simulation-feedback-driven approach and introduces prioritisation mechanisms in the selection of scenarios to explore. In particular, the framework leverages violation prediction models to estimate the likelihood

[1]The project is available at https://github.com/ldegao/ScenarioFuzz-LLM
[2]The project is available at https://github.com/ldegao/TMfuzz
[3]The project is available at https://github.com/yagol2020/SimADFuzz

that a given scenario will lead to violations and, consequently, to steer selection (i.e., sampling) towards more promising cases. This makes the approach attributable to a sampling-based/selection-guided strategy, in which exploration is not uniform but guided by predictive signals [26].

It is important to note that SimADFuzz combines guided selection with scenario transformation operations (e.g., distance-guided mutation strategies to increase vehicle-to-vehicle interactions). However, what most clearly distinguishes it from the other tools is the central role of feedback and prioritisation in deciding which test cases to execute.

3.3 Data Collection

Data collection was designed to ensure comparability across the scenario generators and reproducibility of the measurements, consistently with the defined experimental setup: repeated executions and structured metrics to support probability and risk estimation.

CARLA BehaviorAgent For scenario execution in CARLA, we adopted BehaviorAgent, a driving agent included in CARLA's PythonAPI agent suite. In general, CARLA agents enable a vehicle to follow a route towards a destination and include planning and control logic; they also comply with traffic lights and react to obstacles on the road [27]. The choice of BehaviorAgent addressed two methodological needs:

1. *Fair comparison across generators.* Since the three generators differ in how they explore the scenario space, using a single driving subject reduces an important source of experimental variability. As a result, differences observed in the metrics can be attributed primarily to the generated scenarios, rather than to different control policies.

2. *Relevance in the literature and configurable behaviour.* BehaviorAgent is frequently used as a rule-based baseline in CARLA studies, precisely because it provides deterministic and configurable behaviour that is useful for calibration and comparison against data-driven approaches [28].

Logging System To compute SOTIF-related metrics and reconstruct the outcome of each execution, a data collection system was implemented based on the following components.

The *CarlaBasicLogger*[4] is responsible for producing structured logs in JSON format: it extracts data from a CARLA execution and organises it into the following sections:

- *Metadata.* Contains information about the tool that generated the scenario, the map, and the execution run number for that scenario.
- *Results.* Stores the list of hazards observed during the scenario execution, represented as boolean flags, together with the number of times each hazard occurred.
- *Events.* For each detected hazard, stores information about the frame in which it occurred, the involved actors, and additional hazard-specific details.
- *Mission.* Records the start and end waypoints of the route followed by the ego vehicle during execution.
- *Actors.* Stores the list of spawned NPCs, including their type, role, and spawn waypoint.
- *Frames.* For each frame generated during execution, stores information about the positions of the ego vehicle and NPCs.

At each simulation tick, the *ViolationMonitor*[5] checks whether the ego vehicle is committing observable violations and records them in a structured manner. In particular, the component continuously monitors the ego vehicle to detect speeding, red-light running, stop-sign violations, and off-road events, while applying tolerance thresholds and anti-duplication mechanisms to avoid spurious or repeated detections. Whenever a violation is detected, it records a structured event including frame and

[4]The component is available at https://github.com/NicoUni99997/SOTIF-Compliant, in the `data_gathering` folder

[5]The component is available at https://github.com/NicoUni99997/SOTIF-Compliant, in the `data_gathering` folder

timestamp information, enriched with relevant attributes when applicable, and updates the corresponding run-level flags and aggregated counters stored in the run log and used by the subsequent analysis.

Data Enrichment The JSON file produced by the logger contains detailed information about actors, the ego vehicle trajectory, the mission, and recorded events; however, in its raw form it is not immediately suitable for a systematic comparison across different tools. To make executions comparable and to enable the computation of scenario-level, aggregable metrics, an enrichment phase was introduced to transform the base logs into extended structures, each augmented with an initial set of automatically computed indicators. The enrichment is implemented through independent modules that operate on the log frame-by-frame and store the metrics in the `results` section, while preserving traceability to the original data.

The *criticality analysis* processes the scenario frames and reconstructs the 2D geometry of actors' bounding boxes from the logged vertices, converting them into valid polygons to compute more accurate geometric distances. Based on these reconstructions, criticality indicators are computed from proximity and interaction between the ego vehicle and other dynamic actors, including the minimum distance observed in the scenario and the minimum Time-To-Collision (minTTC).

The *functional analysis* evaluates the ego vehicle's performance with respect to the planned mission by comparing the executed trajectory against a reference route derived from the mission waypoints.

The algorithm estimates route progress by projecting the ego positions onto the planned trajectory and computes the completion rate, lateral deviation statistics, and a route-following stability score. It also computes the actual distance travelled by the vehicle as the sum of distances between consecutive positions, which is useful to distinguish between completed scenarios and scenarios that were interrupted or significantly deviated.

The *kinematic analysis* extracts the ego vehicle speed signals from the log and converts them into longitudinal and lateral components with respect to the vehicle orientation, so as to characterise driving behaviour in the vehicle coordinate frame.

3.4 SOTIF Pipeline

After obtaining the scenarios from the considered generators, each scenario is executed in CARLA through a uniform procedure based on repeated runs (in this work, $N = 10$ executions per scenario). Repetitions are necessary because a single execution is not sufficient to provide robust evidence about hazard occurrence: even under the same initial configuration, simulation may introduce variability due to stochastic or non-deterministic components (e.g., the behaviour of traffic actors). Consequently, repeated execution enables an empirical estimate of how frequently a given hazard manifests within the scenario and supports the computation of aggregated metrics that are less sensitive to run-level noise.

The choice of $N = 10$ represents a practical trade-off between estimate stability and computational cost. With $N = 10$, empirical probabilities are estimated with a granularity of 0.1 (i.e., $P_h \in \{0, 0.1, \ldots, 1\}$), which is sufficient to distinguish scenarios in which hazards are occasional from scenarios in which they are systematic and reproducible, while preserving experimental feasibility under time and resource constraints (including the adopted per-run timeout).

Each run produces a "base" JSON log through the logging system and, subsequently, the same log is enriched by the criticality, functional, and kinematic analysis modules, which store the computed metrics within the `results` block of the log. Once this phase is completed, the SOTIF pipeline is initiated.

ODD and Triggering-Conditions Computation We compute, for each scenario, a set of indicators related to the Operational Design Domain (ODD) and to Triggering Conditions, and store them directly in the log within the `sotif` block. The computation combines two sources:

1. the scenario description (environment, traffic, mission, etc.);
2. selected enriched metrics already available in the log (e.g., min_TTC, MDBV, the ego vehicle's maximum/mean speeds).

The output is twofold: an in-place update of the JSON files and the generation of a summary containing `odd_env`, `odd_infra`, `odd_traffic`, `odd_operational`,

`odd_global`, and the list of triggering conditions for each scenario.

Empirical Probability, Severity, Residual Risk, and the Role of CARLA Leaderboard 2.0 We consider for each scenario, the empirical probability P_h, the severity S_h, and the residual risk R_h for each hazard h, according to the following relation:

$$R_h = P_h \cdot S_h$$

For each hazard h, the script counts how many runs (out of N total runs for the scenario) contain at least one occurrence of that hazard and divides by N, thus obtaining P_h.

The considered hazards include collisions distinguished by category (*collision_vehicle*, *collision_pedestrian*, *collision_static*) and traffic violations (*red_light*, *stop_sign*, *off_road*, *lane_invasion*). This collision breakdown is consistent with the classification performed during logging, where collisions are categorised based on the type of impacted actor (vehicle, pedestrian, or static object).

S_h is treated as a hazard-specific constant weight, defined through the penalty coefficients of CARLA Leaderboard 2.0, which ranks infractions by severity and assigns corresponding coefficients.

Table 3.1: Values associated with events

Event	Value
collision_pedestrian	0.50
collision_vehicle	0.60
collision_static	0.65
red_light	0.70
stop_sign	0.80
lane_invasion	0.85
off_road	0.90

The analysis applies these weights directly to construct S_h for each hazard. For hazards not directly covered by the Leaderboards (`off_road` and `lane_invasion`),

a minimal extension of the weighting scheme is adopted, preserving the assumption of a fixed severity per event type.

Once P_h has been estimated for each scenario, the value of R_h is computed as the product of the empirical probability and the severity.

UMAP, Coverage, and Entropy Computation The run-level vector includes features describing: ego dynamics (e.g., *mean_speed*, *max_speed*, longitudinal accelerations), criticality (e.g., *min_ttc*, *mdbv*, *tet_total*), hazard/violation counts (e.g., *collision_count*, *red_light_count*, *lane_invasion_count*, *off_road_count*), functional indicators (e.g., *completion_rate*, *actual_distance_traveled*, *max_progress_reached*), and SOTIF variables (e.g., *odd_global*, *tc_count*).

These feature vectors serve as input for dimensionality reduction (UMAP) and clustering (K-means) analyses and, subsequently, for measures such as coverage and entropy, which quantify how the scenarios generated by a tool are distributed across different regions of the feature space.

Final report For each scenario, the Hazard Rate HR_h is computed as the fraction of runs (out of N) in which hazard h occurs at least. Severity S_h is treated as a hazard-specific constant weight, while the hazard-level risk is computed as $R_h = HR_h \cdot S_h$.

In addition to hazard-level measures, the script also computes scenario-level aggregates: HR_{avg} and R_{avg} as the mean across the seven considered hazards, the average execution time $T_{\text{exec\_avg}}$ as the mean of `total_simulation_time`, and the `completion_rate` as the proportion of completed runs.

Integration of the Framework into the Selected Tools The analysed scenario generators (ScenarioFuzz-LLM, TMFuzz, and SimADFuzz) were not originally designed, in their respective implementations, to produce complete and uniform logs that meet the requirements of an empirical comparison based on SOTIF-aligned metrics. For this reason, the tools' codebases were extended by introducing interception points within the main execution phases, so as to transparently route scenario- and run-level information to the logging system described in Section 3.3. This integration enabled the consistent collection, across all tools, of data related to the ego vehicle mission,

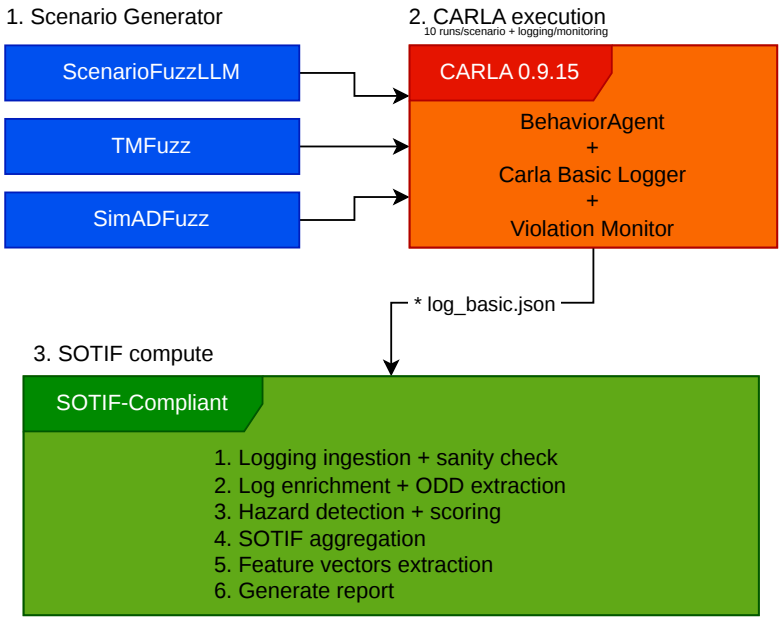

Figure 3.1: End-to-end experimental workflow.

dynamic actors, frame-by-frame state, and critical events/violations, making the produced logs directly compatible with the enrichment phase and the subsequent SOTIF pipeline.

It is important to note that these changes concerned only integration and logging aspects, without altering the scenario generation logic of the frameworks, which was kept unchanged with respect to their original versions.

In the case of ScenarioFuzz-LLM, in addition to the modifications common to the other tools, the language model used as the generation-guidance component was replaced, switching from GPT-4-turbo to Gemini-2.5-pro, while keeping the prompting logic and the output parsing/validation mechanisms unchanged.

Execution environment and experimental configuration The simulations were executed locally on a machine running Ubuntu 22.04, with an Intel i5-12500 CPU, 32 GB of DDR4 RAM, and an NVIDIA GeForce RTX 3060 Ti GPU. At the experimental level, no global time budget was enforced; instead, a per-run timeout of 60 seconds was adopted to bound potentially non-terminating executions and to make execution times and stability comparable across tools.

3.5 Evaluation Methodology

To compare scenario generators in a sound manner, a clear, repeatable, and tool-agnostic methodology is required, capable of measuring both the safety relevance of the produced scenarios (i.e., their ability to elicit adverse events) and the quality of the evidence obtained through repeated executions.

To this end, the evaluation is based on hazards recorded in a structured manner within the logs and subsequently aggregated at the scenario level, so as to make different tools comparable independently of how they generate scenarios.

Metrics The baseline metric used to describe scenario safety is the observation of a set of hazards/violations recorded in the log of each individual run. Additionally, some violations also include boolean flags, which are useful for quick checks and for compatibility with downstream tools.

Table 3.2: Hazards with descriptions and reference values

Hazard	Description	Allowed value
collision_vehicle	Collision between the ego vehicle and an actor of type vehicle.	≥ 0
collision_pedestrian	Collision between the ego vehicle and an actor of type pedestrian (walker).	≥ 0
collision_static	Collision between the ego vehicle and static objects (barriers, buildings, props).	≥ 0
red_light	Entering a traffic-light controlled area while the signal is red without stopping, detected by the monitor and recorded as an event.	≥ 0
stop_sign	Failure to comply with a stop sign (no stop or insufficient stop), detected via trigger volume and a minimum stop duration.	≥ 0
off_road	Ego vehicle leaving the roadway, recorded as an event with frame and position.	≥ 0
lane_invasion	Lane invasion, recorded when the vehicle crosses lane markings (e.g., solid/dashed lines) and stored as an event.	≥ 0

Beyond the counters, the log also retains event-level details in the `events` section, where each occurrence reports at least the frame and timestamp and, for some events, additional attributes such as collision category or the type of crossed lane marking.

Scores For each generated scenario, multiple simulation runs are executed. Each run produces a log containing kinematic and contextual information, which is used for enrichment, and an `events` section that records the events of interest. Based on these logs, the pipeline computes:

- run-level indicators, used when analysing temporal dynamics and efficiency;
- scenario-level indicators, obtained by aggregating the runs associated with the same scenario;
- tool-level aggregated scores, derived by summarising the scenarios produced by each generator.

Effectiveness in Producing Hazardous Scenarios The goal is to quantify how often, on average, the scenarios generated by each tool produce hazardous events and how risky they are overall. Since each scenario is executed multiple times in simulation, the basis of the computation is the empirical probability that a given hazard manifests

within the same scenario over R executions (runs). Let s be a scenario, $h \in \mathcal{H}$ a considered hazard, and $r \in \{1, \ldots, R\}$ the run index. We define an indicator variable that equals 1 if hazard h is observed in run r of scenario s, and 0 otherwise:

$$I_{s,r}^{(h)} = \begin{cases} 1 & \text{if hazard } h \text{ occurs in run } r \text{ of scenario } s \\ 0 & \text{otherwise} \end{cases}$$

The empirical probability is the fraction of runs in which the event occurs:

$$P_h(s) = \frac{1}{R} \sum_{r=1}^{R} I_{s,r}^{(h)}$$

Starting from $P_h(s)$, to summarise the average hazardousness of a scenario, the average hazard rate $HR_{avg}(s)$ is computed as the mean of the empirical probabilities over the considered hazards $\mathcal{H}$:

$$HR_{avg}(s) = \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} P_h(s)$$

Finally, to introduce a measure that accounts not only for frequency but also for the relative severity of different hazards, the average residual risk is computed. By associating each hazard h with a severity weight w_h (constant and shared across tools), the scenario-level residual risk is defined as a weighted average of the empirical probabilities:

$$R_{avg}(s) = \frac{\sum_{h \in \mathcal{H}} w_h \, P_h(s)}{\sum_{h \in \mathcal{H}} w_h}$$

Diversity of Hazardous Scenarios The goal is to measure to what extent the tools explore different types of hazardous scenarios, rather than only how often they generate hazards. To this end, each observed sample (run-level, in this case) is represented through a feature vector $x_i \in \mathbb{R}^d$ and assigned to a cluster via K-Means in the feature space. Let $c(i)$ denote the cluster label of sample i, K the total number of clusters, and $\mathcal{I}_t$ the set of indices of samples generated by tool t.

$$c(i) \in \{1, \ldots, K\}$$

To quantify diversity at the tool level, two complementary metrics are computed: coverage and normalised entropy. Coverage measures how many types (clusters) are actually reached by the tool. Defining the set of covered clusters as:

$$\mathcal{C}_t = \{k \in \{1, \ldots, K\} \mid \exists i \in \mathcal{I}_t : c(i) = k\},$$

the normalised coverage is computed as:

$$Coverage(t) = \frac{|\mathcal{C}_t|}{K}$$

We also compute the entropy of the tool's sample distribution over clusters, which measures how uniformly the tool distributes its samples across the identified types. Let $n_{t,k}$ be the number of samples from tool t assigned to cluster k and $N_t = \sum_{k=1}^{K} n_{t,k}$ the total number of tool samples. The empirical distribution over clusters is:

$$p_{t,k} = \frac{n_{t,k}}{N_t}, \qquad \sum_{k=1}^{K} p_{t,k} = 1.$$

Entropy is then defined as:

$$H(t) = -\sum_{k=1}^{K} p_{t,k} \log\left(p_{t,k}\right),$$

and its normalised version (in $[0, 1]$) is:

$$H_{\text{norm}}(t) = \frac{H(t)}{\log(K)}$$

Non-Critical Hazards The goal is to compare the tools with respect to their ability to elicit non-collision hazards related to driving violations/instability, specifically:

$$\mathcal{H}_{drive} = \{\, lane\_invasion,\ off\_road,\ red\_light,\ stop\_sign \,\}.$$

Since each scenario s is executed R times, the presence of a hazard $h \in \mathcal{H}_{drive}$ within a scenario is estimated through the empirical probability $P_h(s)$. To build a simple and tool-comparable indicator, for each hazard a scenario is considered "positive" if the event occurs at least once in any of the runs, i.e., if $P_h(s) > 0$. We therefore define:

$$I_s^{(h)} = \begin{cases} 1 & \text{if } P_h(s) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Let $\mathcal{S}_t$ denote the set of scenarios generated by tool t. The percentage of scenarios from tool t in which hazard h occurs is then:

$$PctScenarios(t,h) = 100 \cdot \frac{1}{|\mathcal{S}_t|} \sum_{s \in \mathcal{S}_t} I_s^{(h)} \qquad (h \in \mathcal{H}_{drive})$$

Efficiency Efficiency is interpreted as the ability to elicit a hazard as early as possible during simulation execution. The analysis is run-level and relies on two indicators:

1. hit-rate (how often the hazard occurs);
2. time-to-first-event (TTF) (how early it occurs).

Let $\mathcal{R}_t$ denote the set of all runs generated by tool t. For a hazard h, we define the binary *hit* variable as:

$$hit_h(s,r) = \begin{cases} 1 & \text{if event } h \text{ is observed in run } (s,r) \\ 0 & \text{otherwise} \end{cases}$$

The hit-rate for tool and hazard is then:

$$HitRate(t,h) = 100 \cdot \frac{1}{|\mathcal{R}_t|} \sum_{(s,r) \in \mathcal{R}_t} hit_h(s,r)$$

When the event is present, the time-to-first-event is defined as the minimum timestamp among the events of type h recorded in the run:

$$ttf_h(s,r) = \min\{\tau \mid \text{event } h \text{ is observed at time } \tau\}$$

To avoid distortions due to runs in which the event does not occur, temporal statistics are computed only over positive runs (hits only). Let $\mathcal{R}^+_{t,h} = \{(s,r) \in \mathcal{R}_t \mid hit_h(s,r) = 1\}$. Then:

$$mean\_ttf(t,h) = \frac{1}{|\mathcal{R}^+_{t,h}|} \sum_{(s,r) \in \mathcal{R}^+_{t,h}} ttf_h(s,r)$$

$$std\_ttf(t,h) = \sqrt{\frac{1}{|\mathcal{R}^+_{t,h}| - 1} \sum_{(s,r) \in \mathcal{R}^+_{t,h}} \left(ttf_h(s,r) - mean\_ttf(t,h)\right)^2}$$

Finally, to mitigate the dominant effect of highly frequent hazards, two aggregated variants were considered: `driving_all` and `driving_no_lane`.

Project Repository The source code related to the implementation of the experimental pipeline, including the scripts for logging, enrichment, and SOTIF-metric computation, is available in the project GitHub repository at the following address:

`https://github.com/NicoUni99997/SOTIF-Compliant.git`

The repository also includes execution instructions and the folder structure used to reproduce the pipeline described in this chapter.

CHAPTER 4

Results Analysis

This chapter presents and discusses the results of the experimental study conducted to compare automated scenario generation tools in a simulated environment. The analysis is organised into four subsections, each linked to a specific research question, with the aim of providing a clear and progressive interpretation of the collected evidence. In particular, the chapter examines:

- the generators' ability to produce hazardous scenarios and the distribution of the main hazardousness measures;
- the diversity of the critical scenarios revealed, assessed both visually and through summary metrics;
- the occurrence of non-critical driving-related hazards;
- an efficiency perspective, intended as the speed at which safety-relevant events emerge during simulation execution.

Overall, the chapter aims to relate the observed data to the research questions.

4.1 Effectiveness in Producing Hazardous Scenarios

Distribution of the Average Hazard Rate We analysed how the average hazard rate, computed by considering all executions of each scenario and grouping scenarios by generation tool, was distributed.

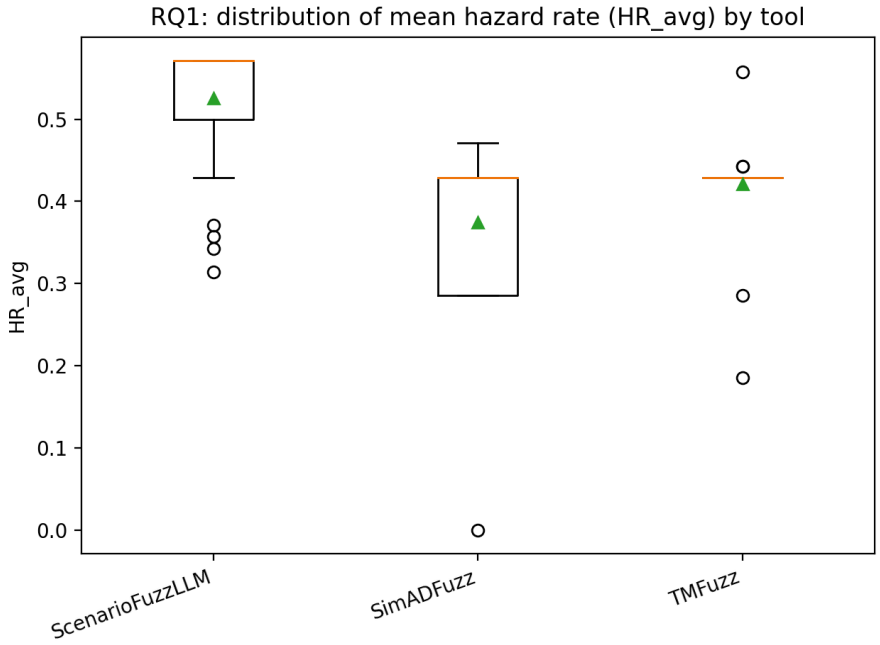

Figure 4.1: Distribution of the average hazard rate.

The boxplot of the HR_{avg} distribution by tool highlighted clear differences among the generators. In particular, ScenarioFuzzLLM exhibited the highest mean values and a higher median than SimADFuzz and TMFuzz, suggesting a stronger tendency to produce scenarios in which hazards emerge more frequently. SimADFuzz overall lay at lower values and shows a wider dispersion, indicating greater variability in the hazardousness of the generated scenarios. TMFuzz tended to fall in an intermediate range, with some outliers indicating the presence of scenarios that are either particularly critical or, conversely, less hazardous.

Distribution of the Average Residual Risk We examined the distribution of the average residual risk by generator, which mirrors the trend of the average hazard rate since severity is defined through constant weights.

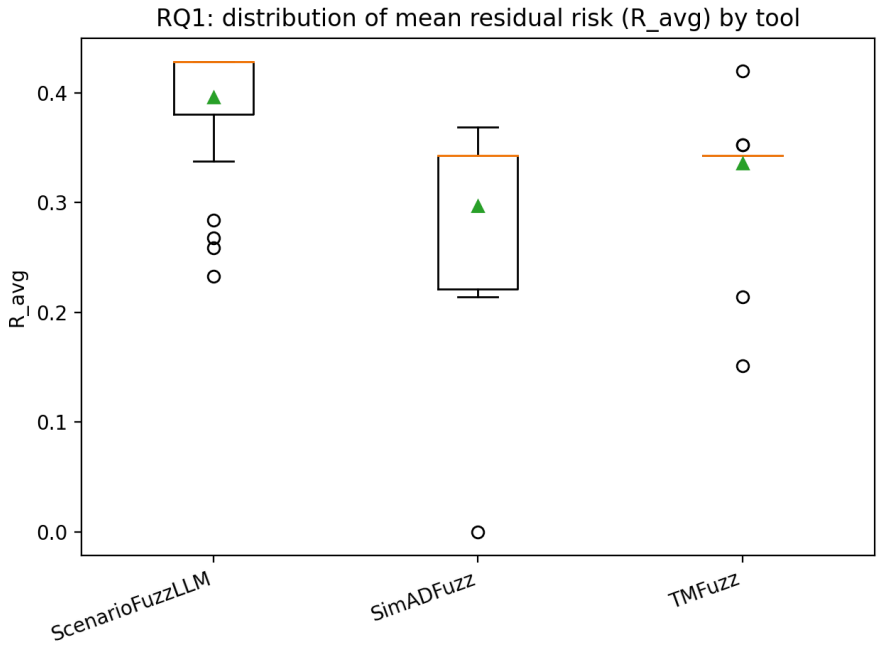

Figure 4.2: Distribution of the average residual risk by generator.

The boxplot of the average residual risk R_{avg} by tool showed a pattern consistent with what was observed for HR_{avg}: ScenarioFuzzLLM tends to report higher mean and median values, whereas SimADFuzz is generally lower and more variable. TMFuzz exhibits intermediate values with a non-negligible dispersion.

Heatmap of Specific Hazards The heatmap of specific hazards makes it possible to examine not only *how much* a scenario is hazardous, but also *how* it becomes so.

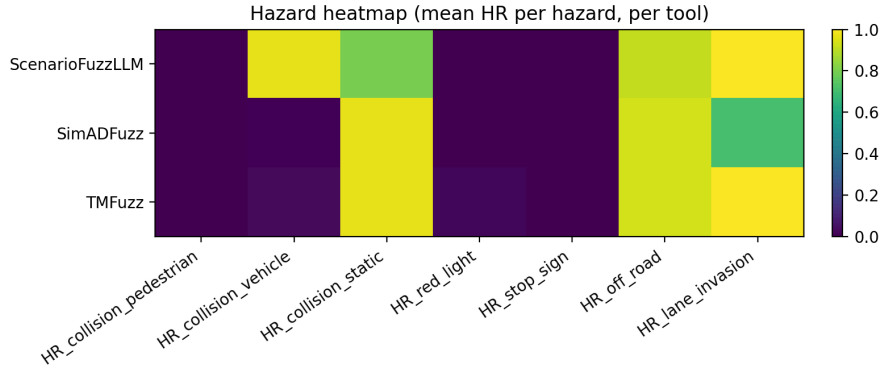

Figure 4.3: Heatmap of specific hazards.

The results indicated that collisions with pedestrians are essentially absent across all tools, suggesting that under the adopted experimental setup this hazard type does not emerge or is not effectively elicited by the tested configurations. Collisions with vehicles, instead, exhibited a strongly differentiated behaviour: *ScenarioFuzzLLM* reported high values, whereas *SimADFuzz* and *TMFuzz* were much lower, which suggests that *ScenarioFuzzLLM* tends to generate scenarios involving hazardous interactions among dynamic agents more frequently. Conversely, static collisions were very frequent for *SimADFuzz* and *TMFuzz*, while *ScenarioFuzzLLM* appeared slightly lower, indicating that these tools more often produce scenarios in which the ego vehicle contacts environmental elements such as barriers, vegetation, or other static obstacles. Both `off-road` and `lane invasion` wew high for all tools, implying that these hazards are easily triggered in the current setup and therefore only weakly discriminative, while still being informative as indicators of lateral instability. Finally, red-light running and stop-sign violations were almost absent, which again suggests that such events are rare or not readily observable within the adopted experimental context.

Conclusions The results indicated that the tools differed markedly in their effectiveness at producing hazardous scenarios. In particular:

- *ScenarioFuzzLLM* emerges as the most effective generator in producing scenarios with higher average hazard rates and higher average residual risk, showing a stronger ability to elicit critical situations in a systematic manner.
- *SimADFuzz* appears overall less effective in terms of average hazard rate and residual risk, while also exhibiting higher variability. This suggests that the criticality of the generated scenarios depends more strongly on specific configurations and does not emerge uniformly.
- *TMFuzz* generally falls in an intermediate position, with a hazard profile more oriented towards static collisions and with the presence of particularly critical scenarios highlighted by the outliers.

👉 **Answer to RQ_1.** The choice of generator significantly affects not only the amount of hazards that emerge, but also the type of criticalities observed, resulting in distinct risk profiles across the analysed tools.

4.2 Diversity of Hazardous Cases

K-Means and UMAP The number of clusters K was selected via silhouette analysis by evaluating multiple values of K.

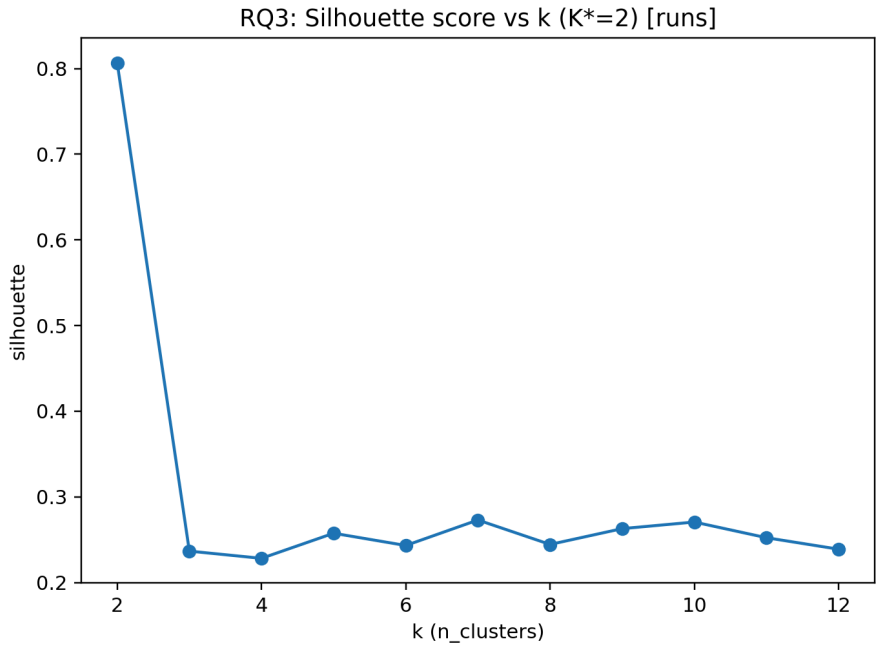

Figure 4.4: Silhouette score analysis.

The silhouette plot showed a pronounced maximum at

$$K^* = 2$$

whereas for higher values the score is noticeably lower. This suggested that, given the data structure in the feature space, the most natural separation of runs was bipartite.

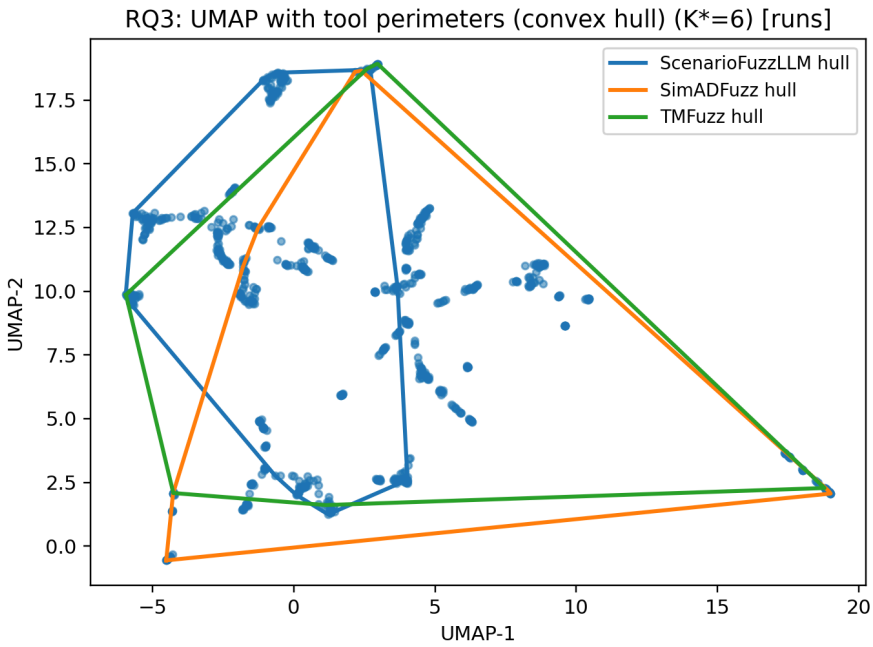

Figure 4.5: UMAP projection.

The UMAP projection provided an intuitive view of how the runs by the different tools are distributed in the embedded space. Adding tool-specific contours highlights that the generators occupy partially different regions and with different extents, supporting the hypothesis that exploratory behaviour is not uniform across tools.

Coverage and Entropy From the run-level coverage and entropy values, clear differences emerged across the three tools. *TMFuzz* achieves the highest coverage, reaching 5 out of 6 clusters (0.83), whereas *ScenarioFuzzLLM* and *SimADFuzz* both reach 4 out of 6 clusters (0.67). This indicated that *TMFuzz* spans a larger number of behaviour types, while the other two tools remain confined to a more limited portion of the observed space.

Table 4.1: Coverage, clusters and Entropy per tool

Tool	Coverage	Clusters	Entropy
ScenarioFuzzLLM	≈ 0.67	4/6	≈ 0.33
SimADFuzz	≈ 0.67	4/6	≈ 0.52
TMFuzz	≈ 0.83	5/6	≈ 0.69

A consistent trend was also observed for normalised entropy: *TMFuzz* attains the highest value (0.69), followed by *SimADFuzz* (0.52), and finally *ScenarioFuzzLLM* (0.33). In practical terms, this means that *TMFuzz* distributes its runs more evenly across the covered clusters, *SimADFuzz* exhibits an intermediate behaviour, whereas *ScenarioFuzzLLM* tends to concentrate a larger share of runs within a smaller number of clusters.

Conclusions Overall, the analysis showed that the tools differed substantially in the diversity of the hazardous scenarios they reveal. Clustering enables the identification of critical behaviour types, and the combination of UMAP visualisation and quantitative metrics highlighted structural differences among the generators.

👉 **Answer to RQ$_2$.** The tools differ in the diversity of the hazardous scenarios they reveal. In particular, *TMFuzz* is the most exploratory and diverse (higher coverage and higher entropy), *SimADFuzz* exhibits intermediate diversity, whereas *ScenarioFuzzLLM* appears more concentrated on recurring hazardous patterns and therefore less diverse in the considered feature space.

4.3 Non-Critical Driving-Related Hazards

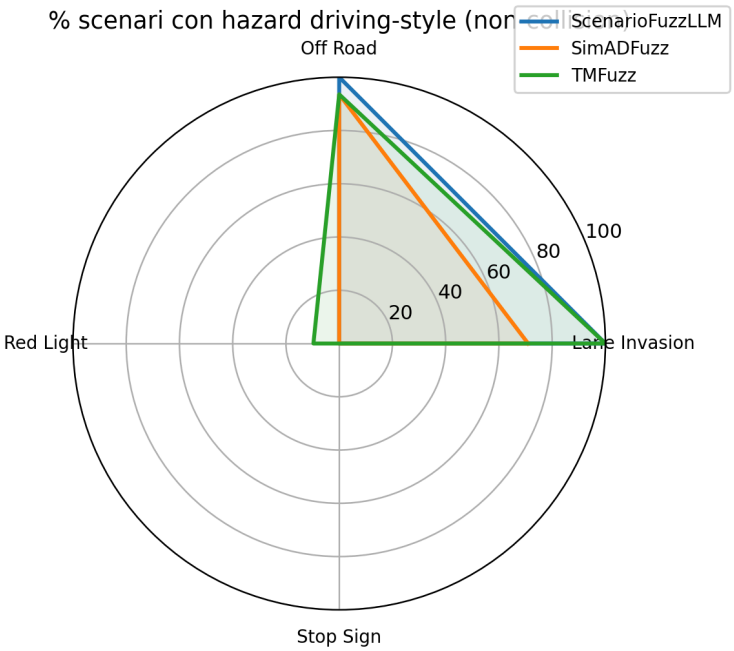

Figure 4.6: Radar chart of non-critical driving-related hazards.

The radar chart summarised, for each tool, the percentage of scenarios in which each driving-style hazard occurs at least once. In particular, we observed that:

- `Off-road` is very high for all tools (≈ 93–100%), indicating that road departure is a recurring event in the generated scenarios and therefore weakly discriminative across generators (while still being informative about a general tendency to produce borderline trajectories).
- `Stop sign` is always 0%: in the analysed dataset this violation does not occur, suggesting that it is rare or not observable under the adopted experimental setup.
- `Red light` is almost absent and appears only for TMFuzz (9.68%), indicating that this tool is the only one that, at least to some extent, elicits scenarios in which this violation is observed.
- `Lane invasion` is saturated (100%) for ScenarioFuzzLLM and TMFuzz, whereas SimADFuzz is lower (70.97%).

The 100% value for `lane invasion` should be interpreted with caution because, in CARLA, this event is often highly sensitive: it can be recorded not only under clearly improper behaviour, but also when the ego vehicle crosses lane markings during common manoeuvres (e.g., turning at intersections, crossing markings near junctions, or trajectories that slightly "cut" the curve). As a result, `lane invasion` tends to become frequent and weakly discriminative, and it may "dominate" the analysis if treated on par with rarer violations (e.g., `red light/stop sign`). For this reason, in the tool comparison, lane invasion is considered useful as an indicator of lateral instability/lane-keeping adherence, but not as the sole discriminator of the quality or severity of the generated scenarios.

Conclusions Overall, the results indicated that the tools generated partly different "improper driving" profiles, but that discrimination across generators is limited by the high recurrence of certain hazards.

1. **Lane invasion is not discriminative in this setup.** The 100% value for *ScenarioFuzzLLM* and *TMFuzz* suggests that this event is saturated and therefore weakly informative for distinguishing the tools. This behaviour is consistent with the highly sensitive detection in simulation, which causes `lane invasion` to appear almost systematically. Consequently, `lane invasion` should be interpreted more as an indicator of lateral instability than as a rare violation.

2. **Off-road is high for all tools (**$\approx$ 93–100%**).** This confirms that the generated scenarios often bring the ego vehicle into borderline trajectories or beyond the drivable surface. Also in this case, the hazard is only moderately useful for differentiating the generators, because the incidence is high across the board.

3. **Meaningful differences emerge on rare hazards.** `Red light` appears only for *TMFuzz* (9.68%), while `stop sign` never occurs. This implies that, under the adopted experimental conditions, the scenario suites produced by the tools rarely trigger traffic-light or stop-sign violations, and that the only distinctive signal along this dimension is *TMFuzz*'s ability to generate some red-light running cases.

👉 **Answer to RQ$_3$.** *TMFuzz* is the most distinctive with respect to this group of hazards because it is the only tool for which red-light events occur in a non-zero fraction of scenarios, whereas *ScenarioFuzzLLM* and *SimADFuzz* exhibit a profile more concentrated on the most common events.

4.4 Efficiency

How often hazards occur The plot showed that `collision` was very frequent for all tools.

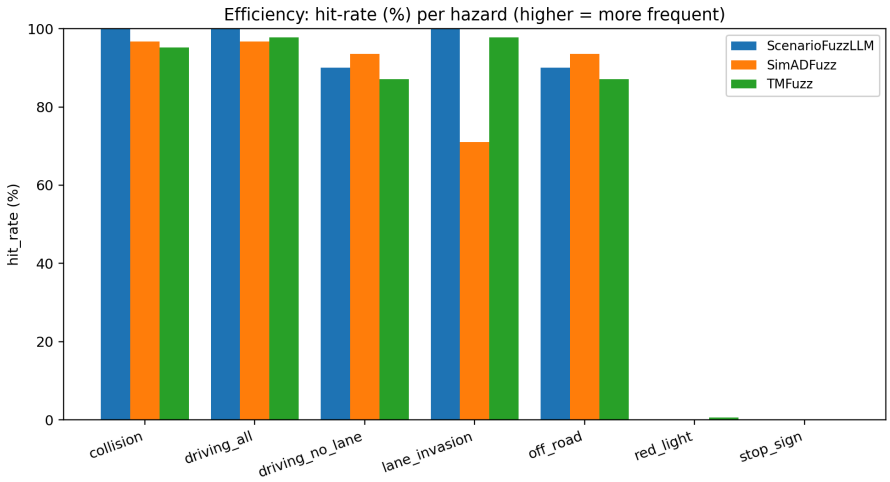

Figure 4.7: Hit-rate plot.

In particular, *ScenarioFuzzLLM* appeared to be the most consistent, with a hit-rate close to 100%, whereas *SimADFuzz* and *TMFuzz* exhibit slightly lower values. Focusing on the driving-style block, `lane invasion` is essentially saturated for *ScenarioFuzzLLM* and *TMFuzz* (approximately 100%), while *SimADFuzz* shows a lower incidence. The `off-road` event is high for all tools (approximately 87–94%), which makes it only weakly discriminative in terms of mere presence. Conversely, `red light` and `stop sign` are almost absent across tools, and therefore contribute only marginally to the interpretation of overall efficiency. Overall, the tools behave similarly with respect to the most recurring phenomena, with *ScenarioFuzzLLM* being marginally more stable on `collision`.

How Early Hazards Emerge? The boxplots and the mean-based plot showed more pronounced differences in terms of timing.

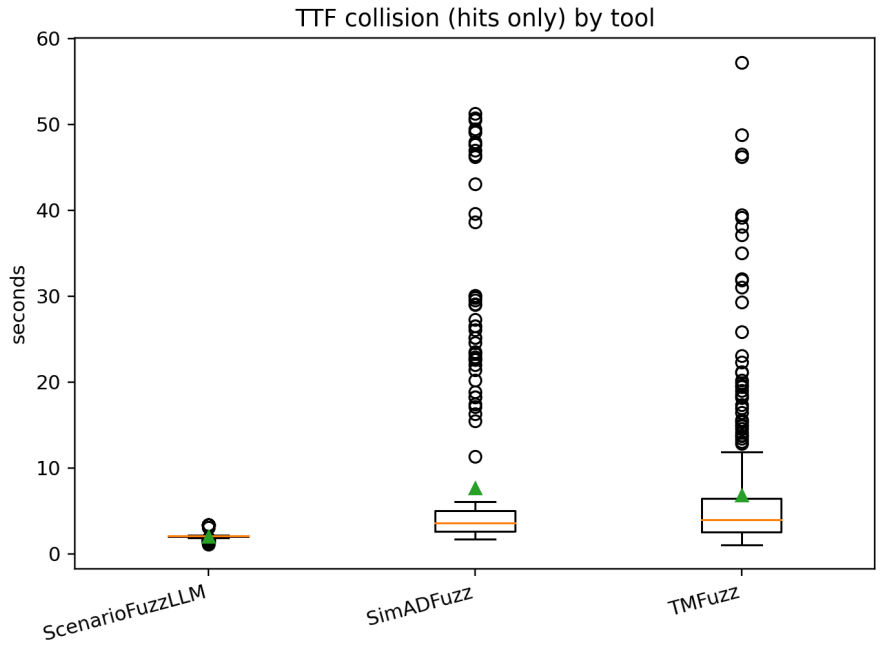

Figure 4.8: Collision time-to-first-event distribution.

The plot highlighted clear differences in the temporal distribution of `collision` events across tools. *ScenarioFuzzLLM* exhibited collisions that occured noticeably earlier and are more concentrated, as reflected by a narrower interquartile range and a lower median, suggesting that when a collision happens it tends to emerge quickly and with higher consistency. In contrast, *SimADFuzz* and *TMFuzz* showed later collision occurrences and, most importantly, substantially higher variability, with long upper tails and numerous outliers, indicating that in a non-negligible number of runs the hazard emerges only after several seconds.

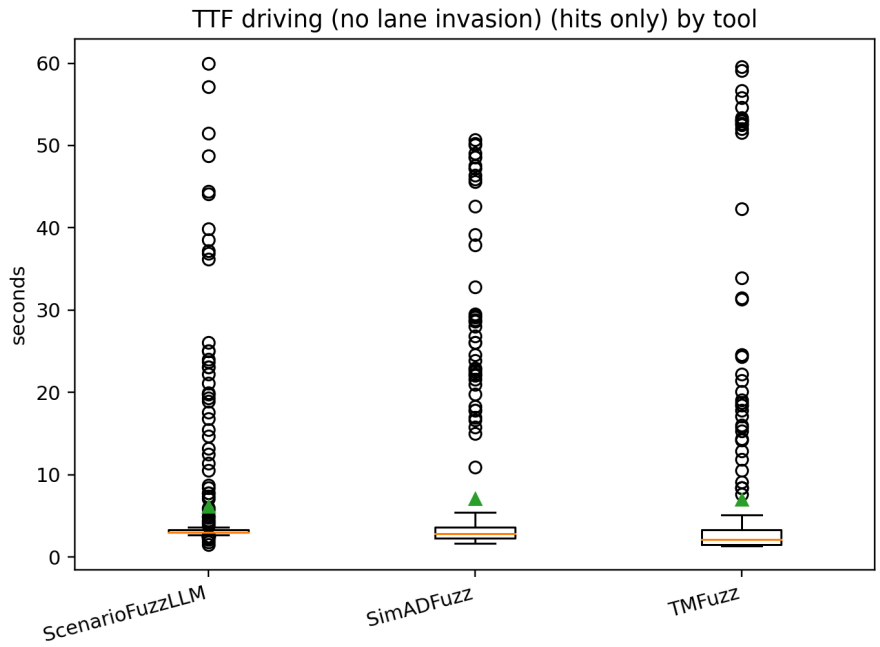

Figure 4.9: Driving-style hazard time-to-first-event distribution.

After excluding `lane invasion`, the differences across tools became less pronounced. The medians are close to each other and all tools exhibit high variability, with several late outliers indicating that driving-style hazards can emerge after a substantial amount of simulated time. Within this general similarity, *TMFuzz* shows a more evident tail of late values, suggesting that in some runs driving-style hazards arise less predictably and only after longer execution times.

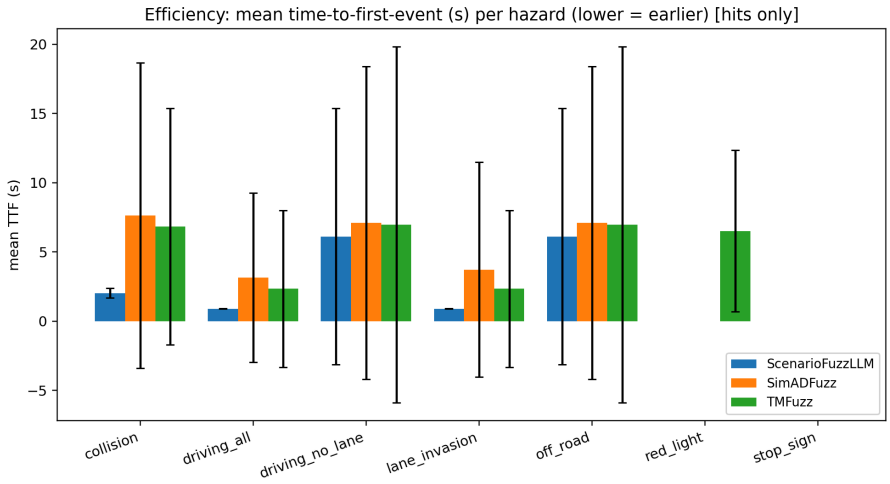

Figure 4.10: Overall distribution of hazard occurrence times.

Conclusions The tools differ in terms of efficiency. In particular, *ScenarioFuzzLLM* is the most efficient at eliciting collision-related hazards, both because collisions occur with high frequency and because they occur earlier on average with reduced dispersion. *SimADFuzz* and *TMFuzz* instead exhibit later and much more variable collisions, indicating a slower and less stable emergence of the hazard. For driving-style hazards (excluding `lane invasion`), differences across tools are more limited and efficiency appears to be driven mainly by the high variability in occurrence times.

👉 **Answer to RQ$_4$.** If the goal is to quickly obtain runs with collisions, *ScenarioFuzzLLM* is the most efficient candidate; if the goal is to observe driving-style hazards, the tools appear more similar and the discrimination is less pronounced.

CHAPTER 5

Threats to Validity

This chapter discusses the main factors that could affect the correctness, reliability, and interpretation of the obtained results. The goal is not to downplay the conducted analysis, but to make explicit the methodological and contextual limitations of the work, clarifying under which conditions the conclusions can be considered robust and under which they should instead be interpreted with caution. The threats are organised according to a taxonomy commonly adopted in empirical research, distinguishing between construct validity, internal validity, conclusion validity, and external validity.

5.1 Construct Validity

Construct validity concerns the alignment between what is intended to be measured and what is actually measured through metrics, tools, and experimental procedures. In this work, the main construct threats are related to the representation of hazards, their interpretation as proxies for "criticality" or "driving style", and the measures adopted for diversity and efficiency.

Interpretative ambiguity of certain criticality hazards Some monitored events, while useful as behavioural indicators, may not unambiguously represent a comparable level of hazardousness across scenarios. In particular, hazards such as lane invasion and, to some extent, off-road can occur with high frequency in simulation and be sensitive to map geometry and generated trajectories. This behaviour may reduce their discriminative power and lead to an overestimation of overall criticality if they are interpreted on par with more severe events such as collisions. To mitigate this risk, results are interpreted by distinguishing between collision-related hazards and driving-style hazards, avoiding assigning the same semantic meaning to events of different nature.

Saturation of lane invasion and dominance in aggregated metrics In the analysed dataset, lane invasion is almost always present, effectively behaving as a "saturated" metric. When a hazard takes values close to the maximum for all tools, it becomes poorly informative for distinguishing generators and may dominate aggregated metrics (e.g., time-to-first-event measures or driving-style profiles). This saturation may stem from the sensitivity of CARLA's detection and from the fact that the event is recorded upon crossing road markings, which may also occur during manoeuvres that are not necessarily anomalous. Consequently, lane invasion is primarily interpreted as an indicator of lateral instability/lane-keeping adherence, rather than as a rare or strongly selective event.

Representing diversity through feature vectors and clustering The diversity of hazardous scenarios is estimated through a feature-vector-based representation and subsequent K-Means clustering, supported by $UMAP$ visualisation. A construct

threat arises from the fact that "diversity" is not a directly observable concept: it depends on the chosen representation (which features are included and how they are normalised) and on the clustering's ability to capture differences that are truly meaningful across scenarios. Therefore, the measured diversity reflects variability within the adopted feature space and does not necessarily cover all possible dimensions of scenario variability (e.g., semantic or contextual aspects not encoded in the features).

Efficiency measured as time-to-first-event Tool efficiency was interpreted as the ability to elicit hazards "earlier" during simulation execution. However, this construct can also be understood in terms of computational cost or total time for generation and execution. In the adopted setup, the overall run duration is bounded by a fixed timeout, making runtime itself only weakly informative. Accordingly, efficiency was measured through time-to-first-event (TTF) and hit-rate. This choice is consistent with the notion of "finding a hazard sooner", but it may not capture other dimensions of efficiency, such as the generator's offline computational cost or orchestration overhead.

Rare hazards and estimate stability For `red light` and `stop sign` events, the observed frequency is very low or null. In these cases, the metric becomes weakly informative and the estimates are inherently unstable or not computable. This situation may reflect limitations of the experimental context (map, actor configuration, spawn points) and may reduce the possibility of comparing tools along these dimensions.

5.2 Internal Validity

Internal validity concerns the extent to which the observed differences across tools can be attributed with reasonable confidence to the tools' characteristics, rather than to external or confounding factors related to the experimental setup, the simulation environment, or the execution pipeline. In the context of this work, the main threats stem from the stochastic nature of simulation, scenario configuration, and the way runs are executed and monitored.

Simulation stochasticity and run non-determinism Even for the same scenario, simulation execution may yield different outcomes due to stochastic or non-deterministic components, such as numerical dynamics, interactions with traffic actors, variability in agent behaviour, and simulator scheduling effects. This variability can affect both hazard occurrence and time-to-first-event, introducing experimental noise that is not directly attributable to the scenario generator. To mitigate this threat, each scenario was executed through repeated runs ($N = 10$), enabling an empirical estimate of hazard occurrence frequency and reducing dependence on potentially anomalous single executions. The choice of $N = 10$ represents a practical trade-off between robustness and computational cost: it allows empirical probabilities to be estimated with a granularity of 0.1 (i.e., $P_h \in \{0, 0.1, \ldots, 1\}$), which is sufficient to distinguish scenarios with sporadic hazards from scenarios with systematic hazards, while preserving experimental feasibility under time and resource constraints (including the adopted per-run timeout). Results were therefore analysed at an aggregated level, using distributions and descriptive statistics (e.g., hazard rate and derived indicators), avoiding conclusions based on isolated single executions.

Actor configuration and initial conditions as confounding factors Run criticality can strongly depend on factors such as the ego vehicle's initial position, traffic spawn and density, non-ego actors' trajectories, and map layout (e.g., proximity to road edges or obstacles). Small differences in initial conditions may lead to substantially different hazards, especially for sensitive events. The comparison was conducted by keeping the simulation environment constant and applying the same execution and monitoring pipeline across all tools. In addition, the evaluation was carried out over a set of scenarios generated by the tools under comparable conditions, and results were aggregated across multiple scenarios and runs to reduce the influence of individual cases.

Fixed timeout and censoring of late events Each run execution is bounded by a maximum timeout. This may introduce a form of censoring: events that would occur beyond the maximum duration are not observed. As a consequence, hazard frequencies and occurrence times may be underestimated for scenarios in which

the event tends to manifest late. For this reason, efficiency was measured through time-to-first-event (TTF) and hit-rate metrics, focusing on events actually observed within the common time budget. Moreover, for the timing analysis, only hits were considered, avoiding artificial distortions that would arise from assigning timeout values to non-events.

Sensitivity and consistency of event logging/monitoring The metrics depend on the produced logs and on the monitors used. An internal threat is that logging errors or inconsistencies may affect the measurements (e.g., out-of-range timestamps, duplicate events, highly sensitive detections near road markings). In particular, hazards such as lane invasion may be easily triggered and therefore saturate some analyses. A uniform logging and normalisation pipeline was adopted across all tools, reducing the likelihood of systematic bias due to instrumentation differences. In addition, consistency checks on timestamps were applied during the timing analysis, and potentially dominant hazards were interpreted with caution.

5.3 Conclusion Validity

Conclusion (or statistical) validity concerns how sound the inferences drawn from the data are and the extent to which results may be affected by sample size, variability, analytical choices, and estimate stability. In this work, the main threats are related to the number of observations per scenario, the distribution of the metrics, and the presence of rare events.

Number of runs per scenario and estimate stability The analysis relies on a finite number of executions per scenario, namely 10 runs. While this partially reduces the impact of non-determinism, it does not necessarily guarantee full convergence of the estimates, especially for hazards with high temporal variability or sporadic occurrence. As a result, some differences across tools may be amplified or attenuated by residual variability. The results were not interpreted based on individual observations, but through aggregations and distributions, reporting dispersion measures and, where appropriate, hit-rates and occurrence counts. This reduces the risk of

conclusions driven by random fluctuations.

Rare events: unreliable or uninformative estimates Some hazards are rare or absent. In these cases, estimates are inherently unstable or not computable, and direct comparisons across tools may become misleading. With very few observations, even the standard deviation is of limited interpretability. Rare hazards were reported for completeness, but they were not used as the primary basis for comparative conclusions. When present, they were accompanied by `n\_hits` and interpreted as limited evidence within the adopted experimental context.

5.4 External Validity

External validity concerns the extent to which the obtained results can be generalised beyond the considered experimental context. In this work, the conclusions derive from a comparison conducted within a specific simulation environment, using a limited set of tools, hazards, and operating conditions; therefore, extending the results to different contexts should be done with caution.

Number and type of considered tools The study compares a limited number of generators, namely three tools. Although representative of different approaches, this set does not cover the full landscape of available tools nor all possible implementation variants (e.g., alternative search-based generators, RL-based approaches, generators grounded on real-world scenarios, or ODD-specific libraries). Consequently, the comparative conclusions cannot be automatically extended to other unanalysed solutions. The results are presented as a comparison among the selected tools under the adopted experimental conditions. In addition, the main generalisable contribution is the evaluation framework and the replicable methodology, which can be applied to other generators in future work.

Dependence on the simulator and the CARLA context The observations are based on simulations conducted in CARLA. Some hazards (e.g., lane invasion/off-road) and their frequencies may depend on the map representation, road geometry, and

simulator detection mechanisms, which may differ in other environments. Therefore, findings such as the saturation of certain events or the absence of specific violations may not replicate identically elsewhere. This work highlights and discusses particularly sensitive hazards and limits their use as primary discriminants in the conclusions. Moreover, the approach remains transferable: given equivalent logging and hazard definitions, the methodology can also be applied to alternative simulators.

ODD and map specificity of the experimental setup Generalisability is limited by the fact that scenarios were generated and executed within a specific Operational Design Domain, on specific maps, and under particular conditions. Rare or absent hazards may be a consequence of the experimental context (road layout, actual presence of signage, routing, traffic) rather than an intrinsic characteristic of the tools. The results are explicitly interpreted as valid for the considered ODD and maps. Any absences/rarities are discussed as an observability limitation of the setup, rather than as a general inability of the generators.

Lack of a real ADS and impact on generalisation The evaluation was conducted in co-simulation with a simulated driving agent rather than with a full real ADS stack (perception–planning–control with real sensors or models). Consequently, direct transferability of the results to real-world scenarios or specific ADS platforms is limited: a real ADS might react differently to the same scenarios, altering the frequency and types of observed hazards. The focus of this work is on evaluating scenario generators and their ability to produce critical conditions in a controlled and repeatable manner. The methodology and metrics remain applicable in the presence of a real ADS, but the numerical results should be considered dependent on the adopted agent.

Limited set of hazards and considered metrics The comparison is based on a defined set of hazards (collisions and violations, driving instability) and on metrics derived from the available logs. Other aspects potentially relevant to safety (comfort, finer near-miss events, advanced kinematic measures, perception-related metrics, more detailed rule compliance) were not included, limiting the generalisa-

tion of “criticality” to what is effectively measured. The hazard set was selected to remain consistent with the evaluation framework and the available logging, while maintaining a clear and replicable definition.

CHAPTER 6

Conclusions and Future Work

This work proposed a replicable evaluation framework aligned with SOTIF principles to compare automated scenario generation tools in simulation, with an empirical application in a CARLA-based environment. Through an end-to-end pipeline consisting of repeated scenario execution, logging, enrichment, and analysis three state-of-the-art generators (ScenarioFuzz-LLM, TMFuzz, and SimADFuzz) were compared to assess effectiveness, diversity, and efficiency, intended as the speed at which safety-relevant events emerge during simulation.

Overall, the results highlight clear differences across the tools. In terms of effectiveness, ScenarioFuzz-LLM generates, on average, more critical scenarios, with higher values for both the average hazard rate and the average residual risk. Beyond magnitude, the hazard-profile analysis indicates that the generators tend to elicit different types of criticalities: ScenarioFuzz-LLM is more strongly associated with collisions involving dynamic actors, whereas SimADFuzz and TMFuzz more often lead to collisions with static elements. From a diversity perspective, TMFuzz appears to be the most exploratory tool, as it reaches a larger number of clusters in the adopted feature space and distributes runs more uniformly; ScenarioFuzz-LLM, conversely, is more concentrated on recurring hazardous patterns and therefore less diverse, while SimADFuzz exhibits an intermediate profile. The UMAP visualisation supports

this interpretation by showing different occupancy patterns in the embedded space across tools. With respect to non-critical driving-style hazards (lane invasion, off-road, red light, stop sign), the comparison is constrained by the strong recurrence of lane invasion and off-road under the adopted experimental setup, which makes these events weakly discriminative across generators. Differences emerge mainly on less frequent hazards: for instance, red light appears only for TMFuzz, whereas stop sign is absent for all tools in the analysed context. Finally, when considering efficiency through time-to-first-event (hits only), ScenarioFuzz-LLM is the most efficient tool for eliciting collisions, with substantially lower mean times and reduced variability compared to the other tools; for driving-style hazards excluding lane invasion, differences are more limited and largely dominated by the high variability of the phenomenon. Taken together, these findings show that the choice of generator affects not only how many hazards are observed, but also which types of criticalities are more likely to emerge and how quickly they manifest.

The main contribution of this work is a concrete and replicable methodology for evaluating and comparing scenario generators in simulation from a SOTIF-oriented perspective. The framework is useful both for researchers and for practitioners who need to select a tool to build effective test suites: depending on the objective, ScenarioFuzz-LLM is better suited to quickly elicit collision-related hazards, whereas TMFuzz is preferable when the goal is to maximise the diversity of hazardous cases. At the same time, the conclusions should be interpreted within the boundaries of the adopted setup: the comparison includes only three tools, depends on CARLA and the considered ODD, does not employ a full real ADS stack, and some metrics are highly sensitive and occasionally saturated, which limits discriminative power for specific event classes.

Future work can extend this study along multiple directions. A first step is to broaden the comparison by including additional generators and parametric variants of the analysed tools, in order to assess the stability of the observed rankings across a wider range of approaches. In parallel, expanding the ODD—by considering different maps, weather/lighting conditions, and traffic densities and compositions—would make it possible to observe hazards that are rare in the current setup and to improve generalisability. Another natural extension is to integrate a real or more realistic ADS

stack (perception–planning–control), to evaluate to what extent the results depend on the simulated driving agent used in this work. The evaluation could also be strengthened by enriching the safety metrics, for instance by adding continuous near-miss measures, finer-grained rule-compliance violations, and comfort/stability indicators, thus capturing criticalities not represented by discrete hazards alone. Moreover, additional sensitivity analyses and more robust sampling strategies would further reinforce the statistical grounding of the conclusions while preserving the overall replicability of the pipeline. Finally, supporting additional simulators or co-simulation backends would allow testing the transferability of the framework beyond CARLA and reduce dependence on simulator-specific behaviours.

Bibliography

[1] National Highway Traffic Safety Administration (NHTSA), "Standing general order on crash reporting for vehicles equipped with automated driving systems (ads) and level 2 advanced driver assistance systems (adas)," https://www.nhtsa.gov/laws-regulations/standing-general-order-crash-reporting, 2021, accessed: 2026-03-14. [Online]. Available: https://www.nhtsa.gov/laws-regulations/standing-general-order-crash-reporting (Citato alle pagine 1 e 7)

[2] S. International, "Sae j3016: Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles april 2021," PDF hosted by UNECE, 2021. [Online]. Available: https://wiki.unece.org/download/attachments/128418539/SAE20J3016_202104.pdf?api=v2 (Citato alle pagine 1 e 7)

[3] D.-G. for Mobility and Transport, "Road safety statistics for 2024: Progress continues amid persistent challenges," *European Commission News*, 2025. [Online]. Available: https://transport.ec.europa.eu/news-events/news/road-safety-statistics-2024-progress-continues-amid-persistent-challenges-2025-10-17_en (Citato a pagina 2)

[4] National Highway Traffic Safety Administration (NHTSA), "A framework for automated driving system testable cases and scenarios," PDF, 2018.

[Online]. Available: https://www.nhtsa.gov/sites/nhtsa.gov/files/documents/13882-automateddrivingsystems_092618_v1a_tag.pdf (Citato a pagina 7)

[5] International Organization for Standardization (ISO) and SAE International, "Iso/sae pas 22736:2021(en) — taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles," https://www.iso.org/obp/ui/, 2021. [Online]. Available: https://www.iso.org/obp/ui/ (Citato a pagina 7)

[6] S. I. R. Practice, "Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles," 2018. [Online]. Available: https://doi.org/10.4271/J3016_201806 (Citato a pagina 8)

[7] International Organization for Standardization (ISO), "Iso 26262-1:2011 — road vehicles — functional safety — part 1: Vocabulary," 2011. [Online]. Available: https://www.iso.org/standard/43464.html (Citato a pagina 8)

[8] ——, "Iso 21448:2022 — road vehicles — safety of the intended functionality (sotif)," 2022. [Online]. Available: https://www.iso.org/standard/77490.html (Citato a pagina 8)

[9] N. Kalra and S. M. Paddock, "Driving to safety: How many miles of driving would it take to demonstrate autonomous vehicle reliability?" *Transportation Research Part A: Policy and Practice*, vol. 94, pp. 182–193, 2016. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0965856416302129 (Citato a pagina 10)

[10] M. R. Cantas and L. Guvenc, "Customized co-simulation environment for autonomous driving algorithm development and evaluation," *arXiv preprint arXiv:2306.00223*, 2023. (Citato a pagina 11)

[11] B. Varga *et al.*, "Optimizing vehicle dynamics co-simulation performance by combining microscopic and mesoscopic traffic simulation," *Simulation Modelling Practice and Theory*, 2023. (Citato a pagina 11)

[12] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "Carla: An open urban driving simulator," 2017. [Online]. Available: https://arxiv.org/abs/1711.03938 (Citato a pagina 12)

[13] BeamNG GmbH, "Beamng.tech – vehicle simulation platform," https://beamng.tech/, 2026. (Citato a pagina 13)

[14] G. B. Till Menzel and M. Maurer, "Scenarios for development, test and validation of automated vehicles," 2018. [Online]. Available: https://www.tu-braunschweig.de/fileadmin/Redaktionsgruppen/Institute_Fakultaet_5/IFR/Dateien_EFS/Publikationen/ScenariosForDevelopment_Test.pdf (Citato a pagina 14)

[15] T. Menzel, G. Bagschik, and M. Maurer, "Scenarios for development, test and validation of automated vehicles," 2018. [Online]. Available: https://arxiv.org/abs/1801.08598 (Citato a pagina 14)

[16] Y. Annapureddy, C. Liu, G. Fainekos, and S. Sankaranarayanan, "S-taliro: A tool for temporal logic falsification for hybrid systems," in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2011. [Online]. Available: https://plv.colorado.edu/papers/staliro-tool-paper.html (Citato a pagina 14)

[17] S. Lin *et al.*, "Tm-fuzzer: fuzzing autonomous driving systems through traffic management and diversity analysis," *Automated Software Engineering*, 2024. (Citato a pagina 14)

[18] S. Lin, F. Chen, L. Xi, K. Xie, Y. Zheng, H. Fei, Y. Sun, and H. Zhu, "Scenariofuzz-llm: Enhancing diversity in autonomous driving scenario fuzzing with llms," in *Proceedings of the 28th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, 2025, pp. 1581–1586. (Citato a pagina 14)

[19] A. Hossain, "A taxonomy and comprehensive survey of scenario generation for autonomous driving: Methods, challenges, and emerging trends in safety-critical testing," Cambridge Open Engage (Working Paper, Version 1),

Sep. 2025. [Online]. Available: https://client.prod.orp.cambridge.org/engage/coe/article-details/68b6f46d23be8e43d68333ff (Citato alle pagine 15 e 19)

[20] C. Xu, W. Ding, W. Lyu, Z. Liu, S. Wang, Y. He, H. Hu, D. Zhao, and B. Li, "Safebench: A benchmarking platform for safety evaluation of autonomous vehicles," 2022. [Online]. Available: https://proceedings.nips.cc/paper_files/paper/2022/file/a48ad12d588c597f4725a8b84af647b5-Paper-Datasets_and_Benchmarks.pdf (Citato a pagina 15)

[21] M. Biagiola and S. Klikovits, "Sbft tool competition 2024 - cyber-physical systems track," in *Proceedings of the 17th ACM/IEEE International Workshop on Search-Based and Fuzz Testing*, 2024. (Citato a pagina 16)

[22] Y. Gao, M. Piccinini, Y. Zhang, D. Wang, K. Moller, R. Brusnicki, B. Zarrouki, A. Gambi, J. F. Totz, K. Storms, S. Peters, A. Stocco, B. Alrifaee, M. Pavone, and J. Betz, "Foundation models in autonomous driving: A survey on scenario generation and scenario analysis," 2025. [Online]. Available: https://arxiv.org/abs/2506.11526 (Citato a pagina 19)

[23] ldegao and contributors, "Scenariofuzz-llm," GitHub repository, 2025. [Online]. Available: https://github.com/ldegao/ScenarioFuzz-LLM (Citato a pagina 20)

[24] S. Lin, F. Chen, L. Xi, K. Xie, Y. Zheng, H. Fei, Y. Sun, and H. Zhu, "Scenariofuzz-llm: Enhancing diversity in autonomous driving scenario fuzzing with llms," in *Proceedings of the 28th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, 2025, pp. 1581–1586. (Citato a pagina 20)

[25] S. Lin *et al.*, "Tm-fuzzer: fuzzing autonomous driving systems through traffic management and diversity analysis," *Automated Software Engineering*, 2024. (Citato a pagina 20)

[26] H. Yang, Y. Zhou, and T. Chen, "Simadfuzz: Simulation-feedback fuzz testing for autonomous driving systems," 2024. [Online]. Available: https://arxiv.org/abs/2412.13802 (Citato a pagina 21)

[27] CARLA Simulator Team, "Carla agents," https://carla.readthedocs.io/en/latest/adv_agents/, 2026. (Citato a pagina 21)

[28] J. Choi and S. Kim, "Predictive risk-aware reinforcement learning for autonomous vehicles using safety potential," *Electronics*, vol. 14, no. 22, p. 4446, 2025. [Online]. Available: https://www.mdpi.com/2079-9292/14/22/4446 (Citato a pagina 21)